AI FIREWALL

GOLDMAN SACHS

One gateway for every AI agent and model.
Every decision **checked**, **logged** and **provable**.

Injection defence · sensitive data redaction · resource access control · known-attack signatures · budgets and loop guards

</> control layer for agentic AI



Team: HotDog for Frog

Goldman Sachs challenge · 42 Prague · October 2026

/already implemented

Prompt injection protection

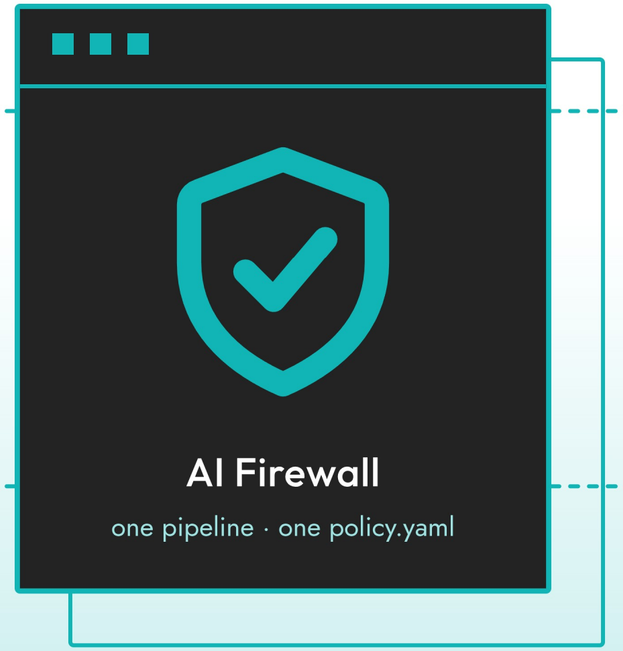
Rules in English, Polish and German, plus an ML classifier. Typed attacks are blocked; poisoned documents are cleaned.

OWASP LLM01

Sensitive data hidden

Emails, cards, IBANs and API keys are redacted. A new regulation is one YAML edit, live.

OWASP LLM02



Resource access control

Default deny. Each folder gets a rule: none, read or write. Agents touch only what you allow.

OWASP LLM06

Historical attacks mitigated

Signatures of known exploits: pickle payloads, eval/exec in tool calls, unsafe model files, jailbreaks.

OWASP LLM03

Every request, reply and tool call is **checked**, **logged** and **explained**: the policy hot-reloads with no restart.

/module system

Every protection is a Control, a plug-in. A new one takes about 15 minutes and never touches the core.

1
One file
`controls/my_control.py`

2
One import line
`controls/__init__.py`

3
One test file
`tests/test_my_control.py`

4
One Policy section
`policy.yaml: mode, params`

ALREADY PLUGGED IN · GROUPED BY MASTER SWITCH

Input validation
`input_validation`

- `injection_heuristics`
- `injection_classifier`
- `attack_signatures`
- `keyword_blocklist`

Data protection
`data_protection`

- `sensitive_data`
- `output_sanitizer`

Resource access
`resource_access`

- `resource_access`

Usage limits
`usage_limits`

- `loop_detector`
- `request_limits`
- `token_budget`

Every Control has a Mode: **off** · **monitor** · **redact** · **block**, set in policy.yaml and applied on the next request.

/any model, one firewall

The agent sends a model name and messages. Only the server Policy picks the adapter and the URL.

Zero code: add a route

Any OpenAI-compatible server, local or hosted, is one entry in policy.yaml. It goes live on save, no restart.

One file: a new protocol

A new provider is one adapter: translate the request, normalize the reply. Chat, proxy and Controls stay unchanged.

Same protection for every model

Adapters only generate. Every model gets the same input and output checks, and the agent can never choose the URL.

policy.yaml

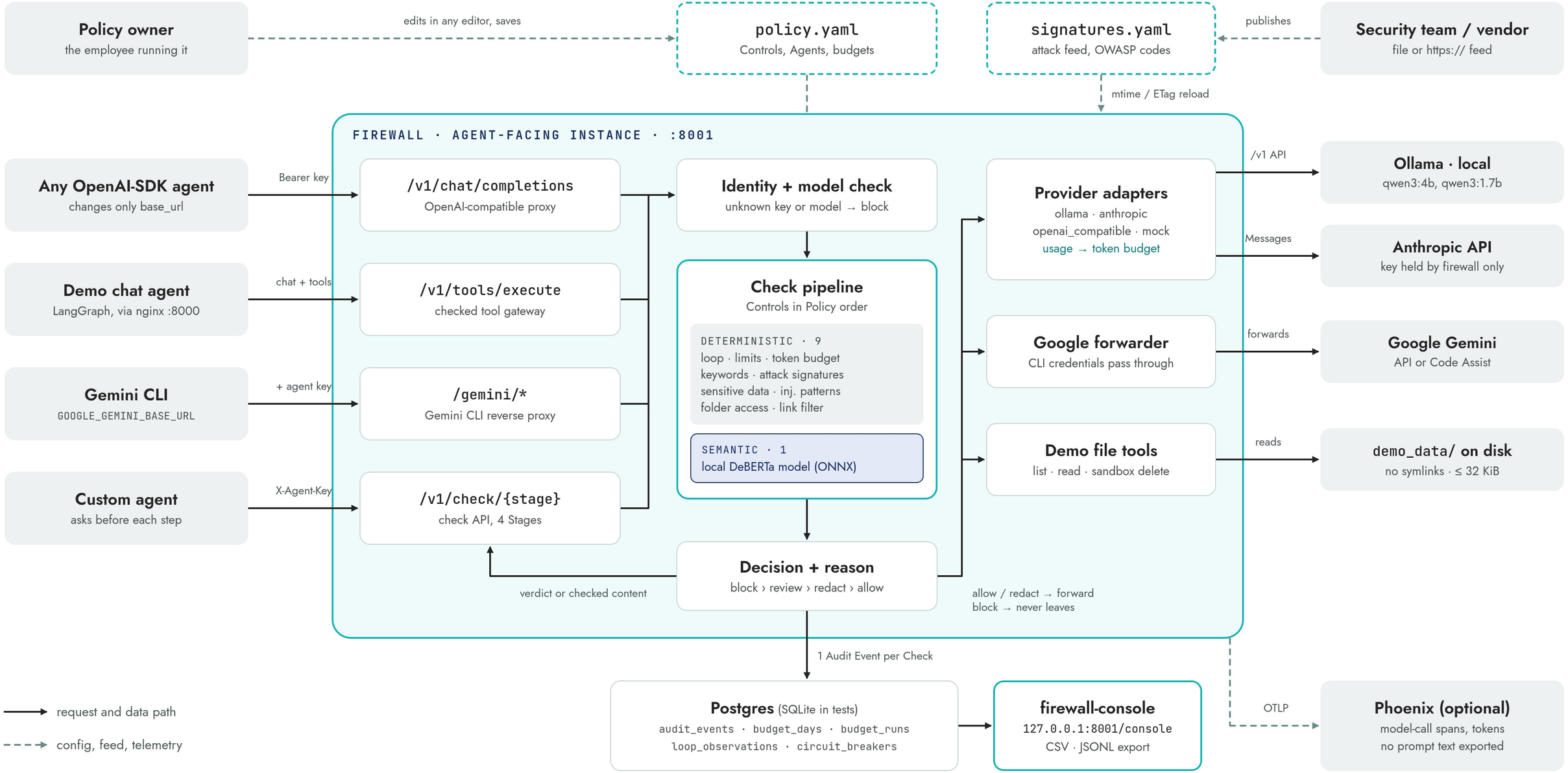
```yaml
upstreams:
  - models: ["qwen3:4b"]
    provider: ollama
    base_url: http://localhost:11434/v1
  - models: ["another-model"]
    provider: openai_compatible
    base_url: https://model-server.example/v1
    api_key_env: MODEL_SERVER_KEY
  - models: ["mock"]
    provider: mock
    base_url: mock://
allowed_models: ["qwen3:4b", "another-model", "mock"]

# the key lives in an env variable, never in the file
```

Supported today: Ollama (local) · Anthropic API · Google Gemini · any OpenAI-compatible server · mock for tests

/full architecture

Four ways in, one pipeline. Nothing reaches a model, a file or a person unchecked, and every Check leaves evidence.



/room to grow

Built to grow along four axes without rewriting the core.

More agents

Each Agent has its own key, Owner and token budget. A new agent is one Policy entry.

More models

A new route is one YAML entry. A new protocol is one adapter file.

More rules

A new Control is one plug-in file. A new regulation is one category in policy.yaml, live.

More traffic

Controls keep no state of their own. Shared state sits in Postgres, so more instances can run side by side.

< 1 ms

rule-based injection check

~30 ms

ML classifier, short prompt, CPU only

NEXT ON THE ROADMAP

- Streaming through the proxy
- Tamper-proof, hash-chained audit log
- M&A ↔ trading information barrier
- Post-hoc threat detection

Fast deterministic checks first, the slower ML model second: **security without a latency penalty.**

/security

Four checkpoints on every agent turn. The firewall sits on both sides of the model and of every tool.



Fails closed

Firewall down or a Control crashes: the Policy decides and model output never leaks. A missing API key blocks the call.

No secrets in the logs

Findings name the pattern and position, never the value. The audit store keeps redacted content by default.

Every denial explained

Each block names the Control and the reason: `Blocked by sensitive_data: card (1)`

Keys stay with the firewall

Provider keys live in env variables on the firewall. An agent holds only its own key and can never choose the model URL.

A Policy you can trust

Edited only by people and validated on every save. A broken file is refused and the last good Policy keeps running.

Local first

Runs on the employee's own PC, bound to 127.0.0.1. The ML model runs locally, and traces never export prompt text.

/ready to work

Not a prototype on slides: it runs today, in three steps.

1 **Start it**

```
docker compose up --build
```

2 **Point your agent at it**

```
OpenAI(base_url="http://localhost:8001/v1",
api_key="<agent key>")
```

3 **Write your rules**

Edit `policy.yaml`: it applies on the next request.

Proven, not promised

- ✓ Full test suite runs offline in one command
- ✓ Works with stock OpenAI SDKs, no code change
- ✓ Policy changes go live with no restart
- ✓ Broken config? The last good Policy keeps running
- ✓ Fails closed: no model output leaks on an error
- ✓ Every decision logged with its rule and reason
- ✓ 11 of 14 stakeholder tests confirmed

Postgres, firewall and chat agent come up together.

/evaluation criteria

Each criterion, and what is already running to back it up.

30% Robustness of the solution and quality of guardrails

10 Controls across 4 Stages. Injection caught by rules (EN, PL, DE, encoded text) plus a local ML model. Card, IBAN and PESEL matches are checksum-verified. Folders are default-deny, and the whole system fails closed.

20% Architecture and performance efficiency

Four entry points, one pipeline. Cheap checks run first and stop early: under 1 ms for rules, about 30 ms for the ML model on CPU. Latency is measured per Control on every Check.

20% Security reporting

One Audit Event per Check in Postgres. Every denial names its Control and reason, with no raw secrets stored. The web console shows posture, threats and budgets, and exports CSV and JSONL.

15% Completeness of the self-testing suite

The full suite runs offline in one command, on a mock LLM. Every Control has allowed and blocked cases for each Stage. 14 stakeholder test cases, 11 confirmed against the brief.

15% Practical implementability and scalability

One `docker compose up`. Existing agents only change base_url. Policy changes go live without a restart. A new Control is one file, and a new model is one YAML entry.

Thank you. **Questions?**